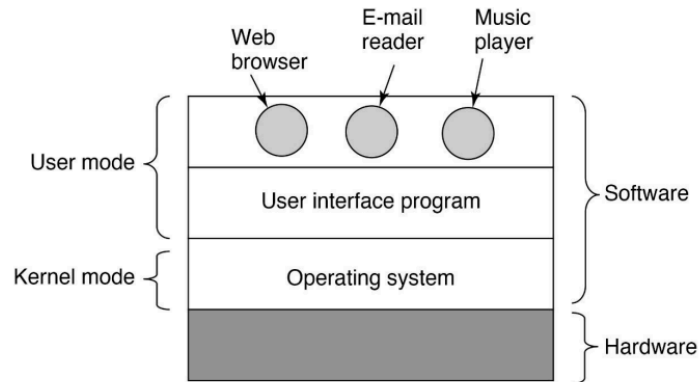


SISTEMI OPERATIVI E RETI DI CALCOLATORI

Sistemi operativi

I **sistemi operativi** esistono poiché rappresentano una soluzione ragionevole al problema di realizzare un sistema elaborativo che si possa impiegare facilmente, per eseguire i programmi e agevolare la soluzione dei problemi degli utenti.

Il sistema operativo è il solo programma che funziona sempre nel calcolatore, generalmente chiamato **kernel** (nucleo).



Oltre al kernel vi sono due tipi di programmi: i **programmi di sistema**, associati al sistema operativo, ma che non fanno necessariamente parte del kernel, e i **programmi applicativi**, che includono tutti i programmi non correlati al funzionamento del sistema.

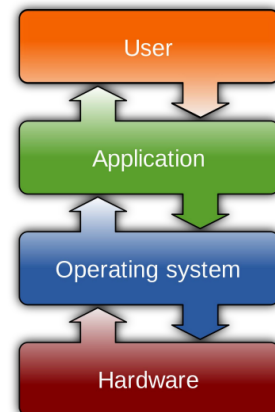
I sistemi operativi mobili sono costituiti esclusivamente da un kernel, ma anche da un **middleware**, ovvero da una collezione di ambienti software che fornisce servizi aggiuntivi per chi sviluppa applicazioni.

Le funzioni principali del S.O. sono:

- la gestione dell'unità centrale di elaborazione (CPU);
- la gestione della memoria volatile (RAM);
- la gestione della memoria di massa (Hard Disk);
- la gestione dell'I/O (Input/Output);
- la gestione dei processi;
- la gestione dei file (file system).

Inoltre, fornisce anche:

- l'interfaccia per l'utente (user, admin o developer);
- l'interprete dei comandi o shell;
- la gestione della sicurezza;
- il controllo accesso alle risorse.



Definizione 5W - 1H del Sistema Operativo

(Who) SO: l'unico programma che funziona sempre nel calcolatore.

(Why) Obiettivo principale: virtualizzazione dell'elaborazione (rendere il sistema più possibile "user-friendly" per gli utenti).

(How) Compito principale : controllo delle applicazioni attive (il funzionamento del S.O. necessita della creazione di questa nuova fattispecie: l'applicazione).

(What) Operazione: computer resource management (le risorse del sistema).

(Where) Generalmente adottato: computer general purpose (elaboratori).

(When) Compleanno ufficiale: 01/01/1970

Tipologie di Sistemi di Elaborazione

Tradizionalmente il computer è visto come una macchina sequenziale: un processore esegue le istruzioni una alla volta, in sequenza, e ogni istruzione è a sua volta una sequenza di operazioni.

Nel tempo si sono affermate diverse tipologie di sistemi di elaborazione:

- sistemi monoprocesso;re;
- mainframe classico;
- personal computer;
- sistemi multiprocessore;
- sistemi distribuiti;
- cluster;
- cloud computing;
- mainframe moderno;
- computer palmari e smartphone;
- sistemi multimediali;
- sistemi di elaborazione in tempo reale;
- sistemi dedicati (embedded system, IoT, OT).

Sistemi Monoprocesso

Un **sistema monoprocesso** dispone di un'unica CPU centrale, che esegue istruzioni di natura generale. Accanto a questa lavorano alcuni processori secondari, ciascuno con un compito particolare, come il controllore del disco, quello della tastiera o il BIOS. Questi processori:

- svolgono compiti specifici;
- eseguono solo un insieme ristretto di istruzioni;
- non eseguono processi utente, che restano di competenza della CPU centrale.

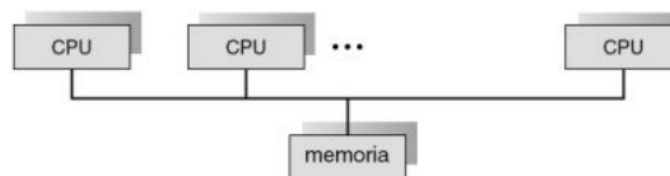
I **sistemi desktop** di oggi permettono un'interazione evoluta con i sistemi centrali, pur svolgendo localmente anche piccole attività di elaborazione. Dispongono di grafica e di dispositivi per l'interazione avanzata e sono spesso sistemi multiprocessore integrati.

Per la gestione delle risorse il sistema operativo usa tre tecniche collegate:

- **Multiprogrammazione**: la memoria viene ripartita tra più processi, che restano caricati contemporaneamente;
- **Multitasking**: la CPU viene condivisa tra questi processi;
- **Time sharing**: è il modo in cui avviene la condivisione. Il tempo della CPU è diviso in piccole fette, assegnate a turno ai processi, così che l'utente abbia l'impressione che vadano avanti tutti insieme.

Sistemi Multiprocessore

Un **sistema multiprocessore** è formato da più processori in stretta comunicazione tra loro. Nei sistemi a processori strettamente connessi (*tightly coupled*), i processori condividono la **memoria**, i **bus** e l'**orologio** (*clock*), e la comunicazione avviene di solito attraverso la memoria condivisa.



Vantaggi dei sistemi paralleli:

- **Maggiore quantità di elaborazione**: con n unità la velocità teorica è n volte quella di una sola (in pratica un po' meno, perché i processori devono coordinarsi).
- **Economia di scala sulle periferiche**: le periferiche sono condivise tra i processori invece di essere duplicate.
- **Maggiore affidabilità**, che si realizza in due modi:

- **Graceful degradation:** il sistema degrada in modo progressivo, cioè le prestazioni calano in proporzione al numero di guasti, senza fermarsi del tutto;
- **Fault tolerance:** il sistema tollera i guasti, ma richiede il riconoscimento del guasto, la diagnosi ed eventualmente la riparazione.

Sistema multiprocessore asimmetrico:

- Ogni processore ha un lavoro specifico assegnato.
- Un processore principale (*master*) organizza e gestisce il lavoro dei processori *slave*.
- L'organizzazione è gerarchica.
- È più comune nei sistemi molto grandi.

Sistema multiprocessore simmetrico (SMP):

- Ogni processore può eseguire tutte le operazioni.
- L'organizzazione è non gerarchica: i processori sono alla pari.
- Molti processi possono girare contemporaneamente senza deterioramento delle prestazioni.
- Serve un bilanciamento del carico tra i processori.
- La maggior parte dei sistemi moderni supporta SMP.

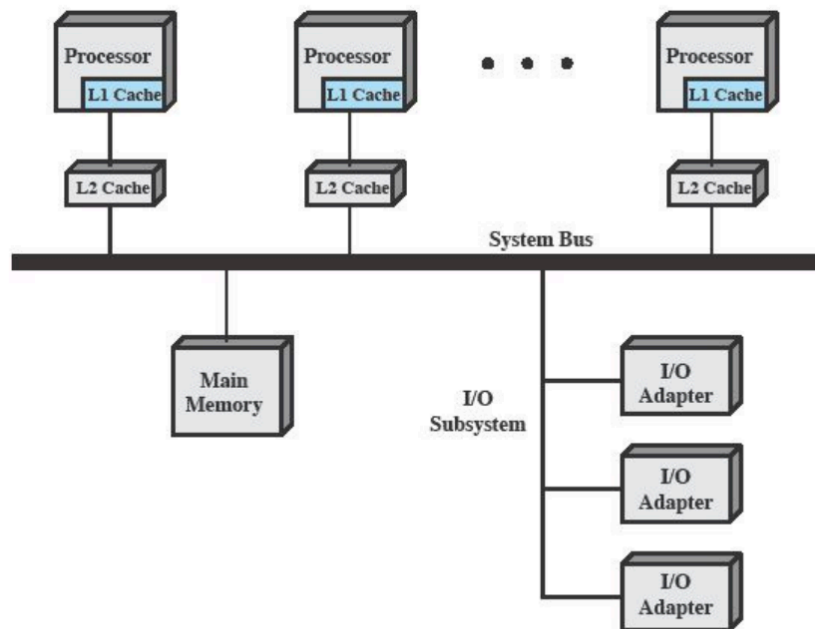


Figure 4.9 Symmetric Multiprocessor Organization

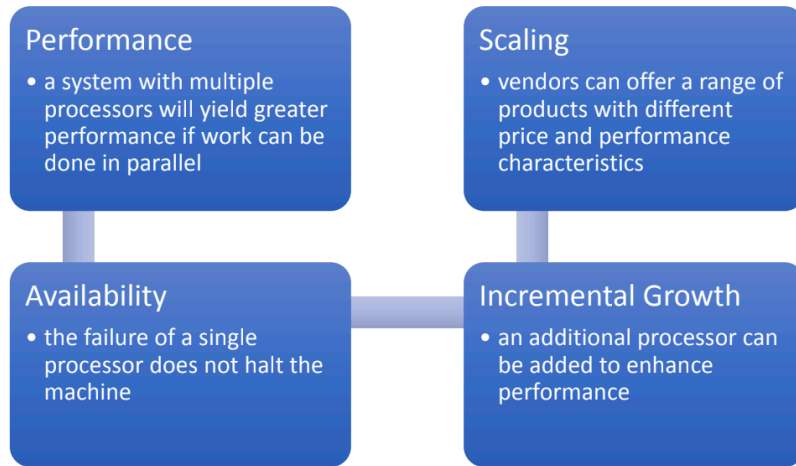
Caratteristiche di un sistema SMP

Un sistema SMP è un sistema di calcolo autonomo (stand-alone) con le seguenti caratteristiche:

- due o più **processori simili** e di capacità comparabile;
- i processori condividono la **stessa memoria principale** e sono interconnessi tramite un bus o un altro schema di connessione interno;
- i processori condividono l'accesso ai **dispositivi di I/O**;
- tutti i processori possono svolgere **le stesse funzioni**;
- il sistema è controllato da un **sistema operativo integrato**, che gestisce l'interazione tra i processori e i loro programmi a livello di job, task, file ed elementi di dati.

I principali problemi di progettazione sono:

- Processi o thread concorrenti simultanei;
- Scheduling;
- Sincronizzazione;
- Gestione della memoria;
- Affidabilità e tolleranza ai guasti.



Computer multicore

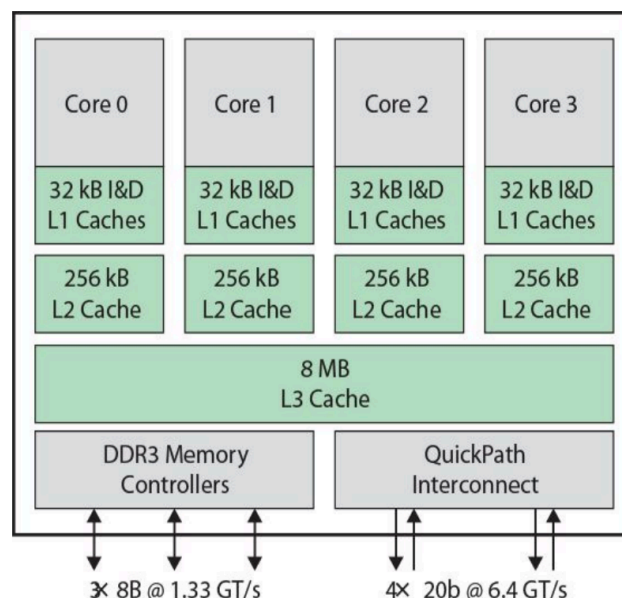
Un computer multicore è detto anche **chip multiprocessor**. Combina due o più processori (chiamati **core**) su un unico pezzo di silicio (**die**):

- Ogni core contiene tutti i componenti di un processore indipendente.
- Inoltre, i chip multicore includono anche la **cache L2** e, in alcuni casi, la **cache L3**.

Es. Intel Core i7

Il chip supporta due forme di comunicazione esterna verso altri chip:

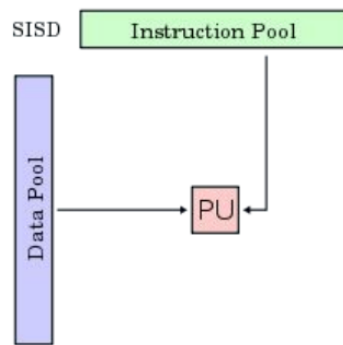
- **Controller di memoria DDR3:**
 - Porta sul chip il controller di memoria per la memoria principale DDR (*double data rate*).
 - Con il controller di memoria sul chip, il **Front Side Bus** viene eliminato.
- **QuickPath Interconnect (QPI):**
 - Permette comunicazioni ad alta velocità tra i chip di processori collegati.



Categorie di sistemi di calcolo (tassonomia di Flynn)

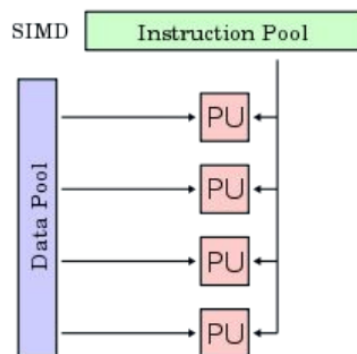
1. **Single Instruction Single Data (SISD):**

- Un singolo processore esegue un singolo flusso di istruzioni per operare su dati memorizzati in un'unica memoria.



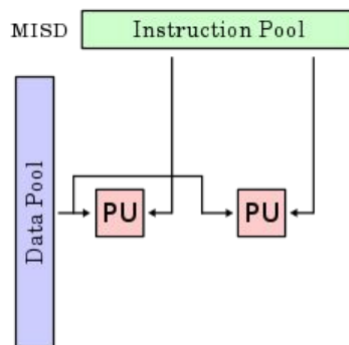
2. Single Instruction Multiple Data (SIMD):

- Ogni istruzione viene eseguita su un diverso insieme di dati dai diversi processori.



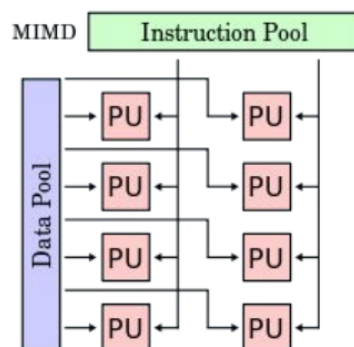
3. Multiple Instruction Single Data (MISD):

- Una sequenza di dati viene trasmessa a un insieme di processori, ciascuno dei quali esegue una diversa sequenza di istruzioni.



4. Multiple Instruction Multiple Data (MIMD):

- Un insieme di processori esegue simultaneamente diverse sequenze di istruzioni su diversi insiemi di dati.



Sistemi Distribuiti

Nei sistemi distribuiti il calcolo viene distribuito tra diversi elaboratori **fisicamente distinti**, che possono essere **eterogenei**.

Questi sistemi sono **sistemi lascamente connessi**:

- Ogni processore possiede una propria memoria locale;
- I processori comunicano tra loro mediante linee di comunicazione, come bus ad alta velocità o linee telefoniche.

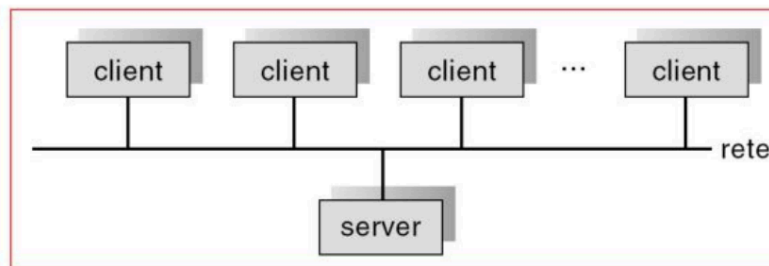
i Vantaggi dei sistemi distribuiti:

- **Condivisione delle risorse**:
 - dati;
 - servizi.
- **Rapidità di calcolo**: distribuzione del carico.
- **Affidabilità**.

I sistemi distribuiti necessitano di un'infrastruttura di rete, che può essere:

- una rete locale (*Local Area Network*, **LAN**);
- una rete geografica (*Wide Area Network*, **WAN**).

Inoltre, possono essere organizzati come sistemi **client-server** o punto-a-punto (**peer-to-peer**), oppure possono sfruttare infrastrutture di **cloud computing**.



Cluster

Un **cluster** è un'architettura formata da più computer fortemente connessi tra loro, realizzata usando computer disponibili sul mercato.

Rispetto ai sistemi SMP ha una capacità di elaborazione superiore e permette di eseguire contemporaneamente un'applicazione su più PC. Per questo richiede la **programmazione parallela**, cioè programmi con componenti eseguibili in parallelo.

Le caratteristiche principali sono:

- **economie di scala sulle periferiche**;
- possono essere **simmetrici** o **asimmetrici**;
- **affidabilità del sistema in caso di guasti**: ogni PC è controllato da almeno un altro PC, che recupera il lavoro in caso di guasto (*Business Continuity*).

Cloud Computing

Il **cloud computing** è un insieme di tecnologie informatiche che permettono di utilizzare risorse **hardware** (ad esempio storage, CPU) o **software**, distribuite e virtualizzate in rete.

Il termine the cloud significa “nuvola” di risorse, le cui caratteristiche non sono note all'utilizzatore.

Si usa con il modello **pay-as-you-go**, cioè si paga in base a quanto si utilizza.

Il cloud computing comprende diversi modelli di servizio:

- **SaaS** (*Software as a Service*);
- **PaaS** (*Platform as a Service*);
- **CaaS** (*Container as a Service*);
- **FaaS** (*Function as a Service*);
- **IaaS** (*Infrastructure as a Service*).

Mainframe classico

Il **mainframe classico** è un'architettura orientata all'elaborazione di **lavori non interattivi** (*job*). E' composto da un processore, una memoria centrale (milioni di gigabyte), numerosi nastri e dischi (circa 1000) e stampanti.

L'elaborazione avviene a **lotti** (*batch*): i tempi di processo si riducono raggruppando i job (processi) in batch con necessità simili. Il sistema esegue numerosi lavori di routine alla volta, con prodigiose quantità di I/O e senza la presenza di alcun utente che interagisca con la macchina.

Esistono due tipi di sistemi:

- **sistemi monoprogrammati**: la CPU è sottoutilizzata;
- **sistemi multiprogrammati**:
 - la memoria centrale è ripartita tra i job (**multiprogrammazione**);
 - la CPU è condivisa (**multitasking**).

Mainframe moderno (HPC)

Il **mainframe moderno** è costituito da **grandi server**, spesso raffreddati a liquido. Supporta molti utenti che operano contemporaneamente ed è alla base dei giganteschi server web centralizzati. E' composto da CPU, memoria centrale, terminali, nastri e dischi, stampanti. Elabora contemporaneamente più flussi di attività (processi) e si occupa dell'elaborazione di transizioni e della condivisione del tempo macchina.

Sono **sistemi multiutente**, che usano tre tecniche:

- **multiprogrammazione**: ripartizione della memoria tra i processi;
- **multitasking**: condivisione della CPU;
- **time sharing**: gestione della CPU in condivisione di tempo.

Sistema dedicato (embedded)

I **sistemi dedicati** sono sistemi di elaborazione pensati per supportare **una sola applicazione**. Ne sono esempi gli elettrodomestici, i sistemi hi-fi, il motore dell'automobile (CANBus), i sistemi biomedicali, le protesi, le carte di credito e così via.

Le loro caratteristiche sono:

- tutto il software è su **ROM**;
- hanno ridotte caratteristiche di prestazioni computazionali, memoria e periferiche;
- hanno spesso caratteristiche di **real-time** e **multitasking**.

Sistemi di elaborazione in tempo reale (real-time)

I **sistemi real-time rispondono agli eventi in tempo reale**, fornendo la risposta nel rispetto di rigorosi vincoli temporali. Si distinguono in:

- sistemi in tempo reale stretto (**hard real-time**);
- sistemi in tempo reale lasco (**soft real-time**);

Hanno un'architettura in grado di scambiare segnali con il mondo esterno attraverso **sensori** e **attuatori**, usano:

- schede di acquisizione segnali (input digitali/analogici);
- schede di attuazione controlli (output digitali/analogici).

Sistemi multimediali

I **sistemi multimediali** dispongono di supporti avanzati per l'**interazione multimediale**. Ne sono esempi i sistemi di controllo delle console per giochi (Nintendo Wii, Microsoft Xbox, Sony PlayStation, ecc.), ma anche PuzzleLinux.

In questi sistemi la trasmissione dei dati deve attenersi a **specifiche frequenze**.

Sono **sistemi interattivi multiprocesso**, che usano tre tecniche:

- **multiprogrammazione**: ripartizione della memoria tra i processi;
- **multitasking**: condivisione della CPU;
- **time sharing**: gestione della CPU in condivisione di tempo.

PDA/Smartphone

I **PDA** (*Personal Digital Assistant*) sono sistemi di elaborazione **portatili e di dimensioni estremamente ridotte**, orientati al supporto di attività personali. Ne fanno parte:

- i sistemi palmari;
- i telefono cellulari.

Sono **sistemi interattivi multiprocesso** con:

- ridotto consumo di potenza;
- basso numero di processi.

Gli **smartphone** ne sono un'evoluzione (sistemi operativi iOS e Android) e hanno:

- processori multi-core;
- interfacce multi-touch;
- prestazioni elevate.

Microkernel

Un **microkernel** è un **piccolo nucleo del sistema operativo** che fornisce la base per estensioni modulari.

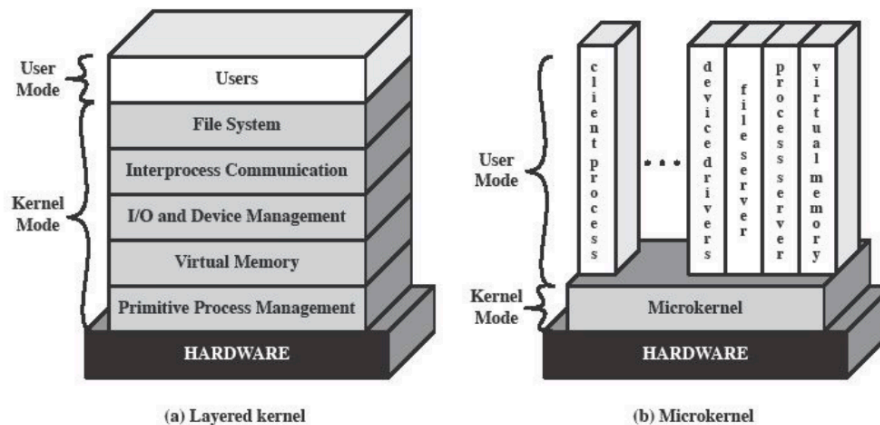


Figure 4.10 Kernel Architecture

Progettazione del microkernel

Gestione della memoria (Memory Management)

- **Gestione della memoria a basso livello**: consiste nel mappare ogni pagina virtuale su un frame di pagina fisica (*physical page frame*).
- La maggior parte dei compiti di gestione della memoria avviene nello **spazio utente** (*user space*).

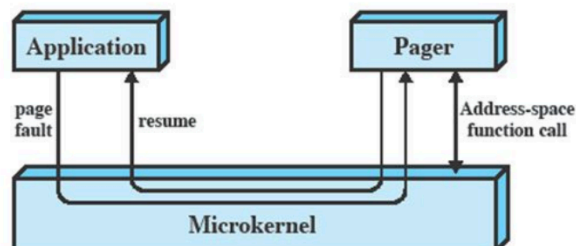


Figure 4.11 Page Fault Processing

Comunicazione tra processi (Interprocess Communication)

In un sistema operativo basato su microkernel, la comunicazione tra processi o thread avviene tramite **messaggi**.

Un messaggio comprende:

- un **header** (intestazione), che identifica il processo mittente e quello destinatario;
- un **body** (corpo), che contiene dati diretti, un puntatore a un blocco di dati oppure alcune informazioni di controllo sul processo.

Gestione di I/O e interrupt (I/O and interrupt Management)

All'interno di un microkernel è possibile gestire gli **interrupt hardware come messaggi** e includere **le porte di I/O negli spazi di indirizzamento**.

- Un particolare processo a livello utente (*user-level*) viene assegnato all'interrupt, e il kernel mantiene la corrispondenza (*mapping*) tra i due.

Vantaggi di un'organizzazione a microkernel

- **Interfacce uniformi** per le richieste fatte da un processo;
- **Estensibilità**;
- **Flessibilità**;
- **Portabilità**;
- **Affidabilità**;
- **Supporto ai sistemi distribuiti**;
- **Sistemi operativi orientati agli oggetti**.

S.O. : Obiettivi

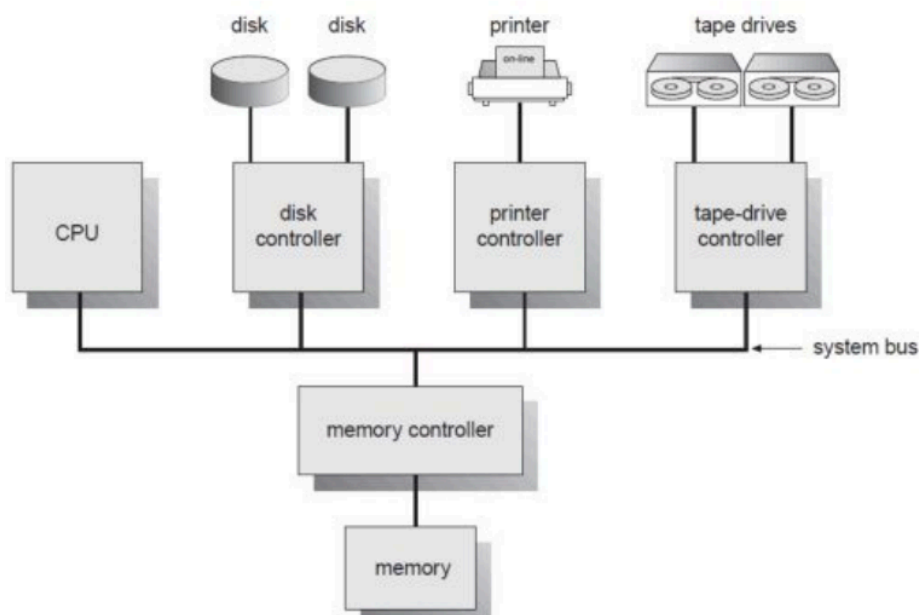
Input/Output

Un calcolatore è composto da una CPU e diversi **controllori di dispositivi**. Sono le risorse condivise:

- **Bus**: mezzo condiviso di comunicazione;
- **RAM**: luogo condiviso di allocazione dei dati da elaborare;
- **CPU**: strumenti condivisi di elaborazione.

Lo spostamento dei dati da/verso la CPU prende il nome di **I/O**:

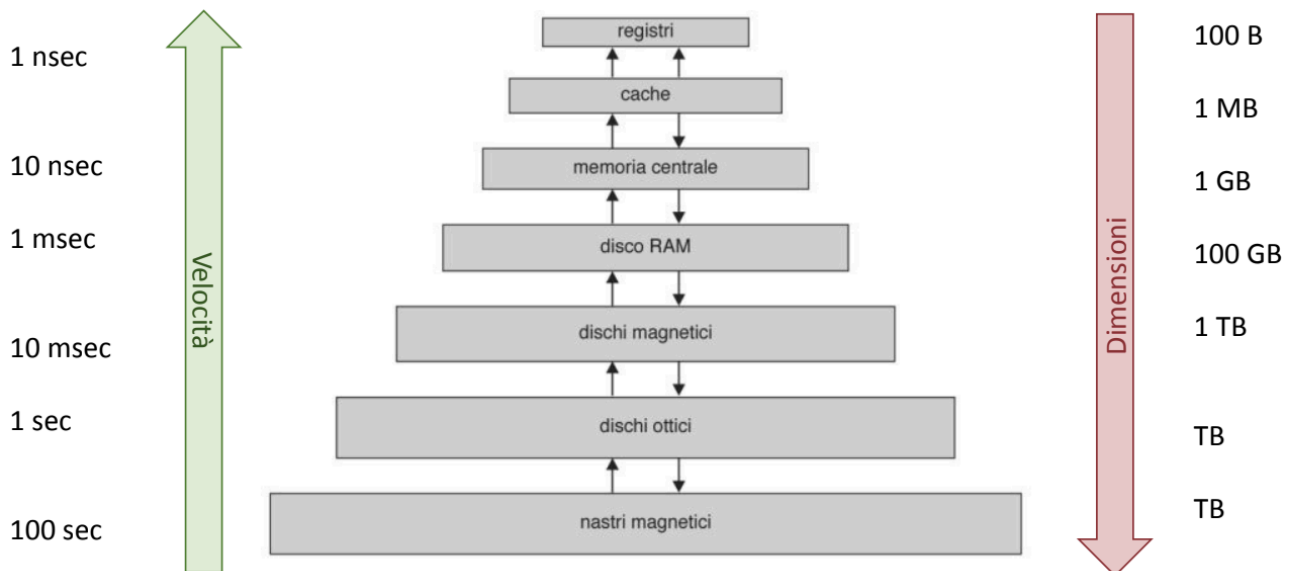
- L'I/O avviene tra il dispositivo e il **buffer locale del controller**;
- La CPU guida poi lo spostamento dei dati (che viaggiano sul **bus**) dai buffer locali dei controller alla memoria, e viceversa.



L'obiettivo è rendere il sistema **quanto meno impattato possibile dalla lentezza dei dispositivi periferici**.

Cache

- La cache di **primo e secondo livello** è specifica per ogni core;
- La cache di **terzo livello** è condivisa tra i core. Questa condivisione è all'origine di vulnerabilità come **Meltdown** e **Spectre**.



Tecniche di I/O

1. I/O programmato;
2. I/O gestito con interrupt;
3. I/O con DMA;
4. I/O con canali (processore separato).

I/O programmato

L'I/O programmato è tipicamente utilizzato da architettura di **fascia bassa** o nel **controllo industriale** con basse necessità di I/O multiplo:

- Per ogni scrittura il processore esegue una **istruzione di I/O**. Per scrivere un blocco di n caratteri, la CPU esegue n istruzioni di output carattere.
- Per ricevere un dato, la CPU legge in un **ciclo stretto** la porta di I/O in attesa del carattere.

L'informazione di input disponibile viene tipicamente affidata a un **bit di controllo**.

Svantaggio: non è ottimale nell'ipotesi in cui l'I/O di quantità di dati relativamente grosse occupi una parte rilevante del tempo macchina. Il microprocessore spenderà la maggior parte del suo tempo in operazioni di I/O.

Interrupt

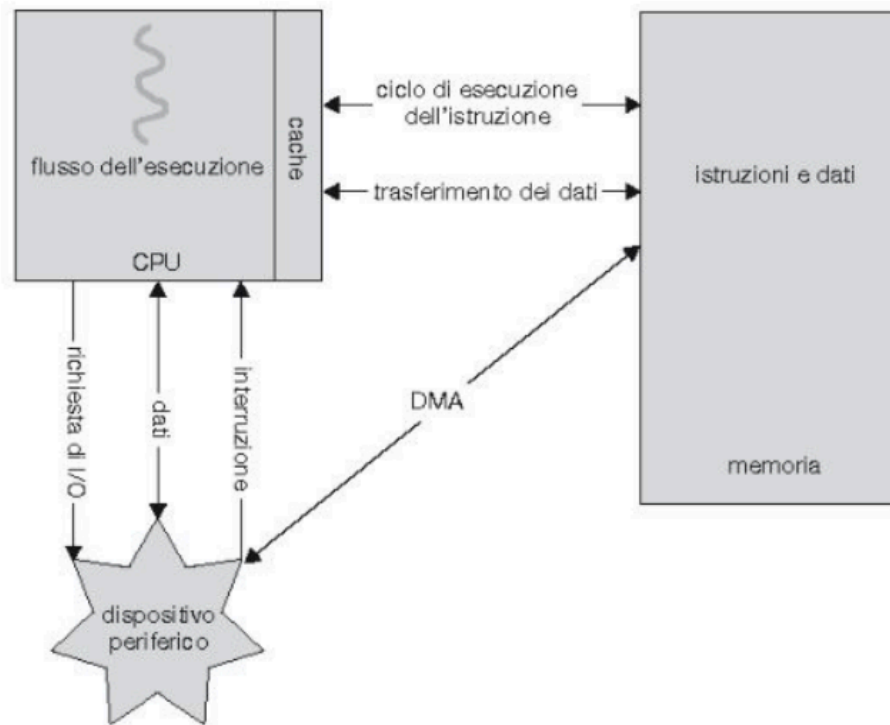
Ogni controllore di dispositivo I/O dispone di una **memoria interna (buffer)**. Per avviare un'operazione di lettura (*read*) di un byte, il controllore:

- copia i dati sul buffer;
- informa il **driver del dispositivo** tramite **interrupt**;
- il driver cede il controllo al sistema operativo, restituendo un **puntatore ai dati nel buffer**.

Questo metodo è **oneroso per grosse dimensioni** (molto byte).

DMA (Direct Memory Access)

Una volta impostato il buffer, il controllore del dispositivo I/O **trasferisce direttamente grandi porzioni di dati** dal dispositivo alla memoria. E' necessario **un solo interrupt**, quindi una sola interazione con la CPU per ogni blocco di byte.



Virtualizzazione

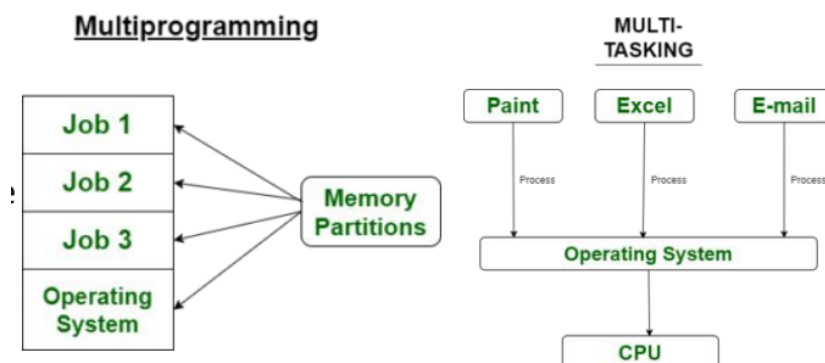
Un **processo** è un programma in esecuzione:

- indipendente dagli altri programmi;
- che non deve interferire con gli altri programmi;
- che trae giovamento dalla gestione ottimizzata delle risorse;
- che deve sottostare all'allocazione e alla condivisione ordinata delle risorse rispetto a spazio e tempo.

L'obiettivo è rendere il sistema quanto più possibile "**dedicato**" all'esecuzione di ogni processo.

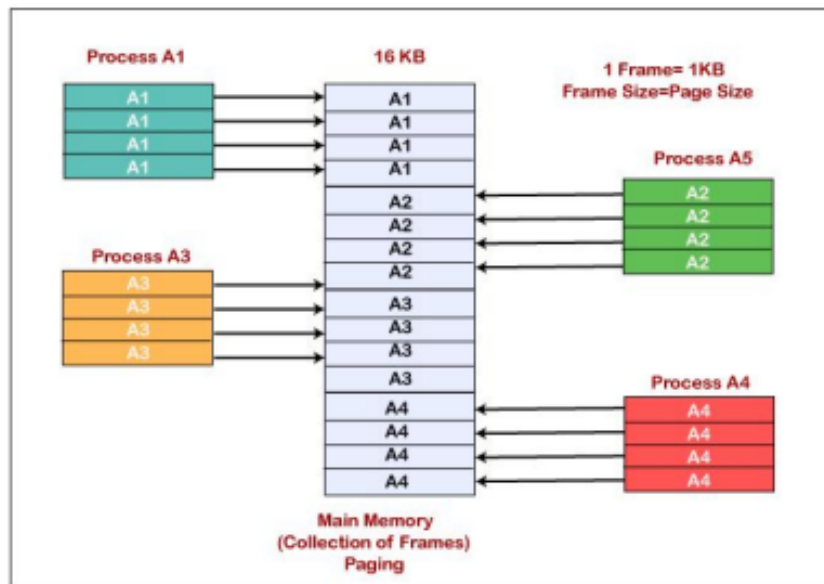
Condivisione temporale della CPU

- **Elaborazione batch**: un processo alla volta, dall'inizio alla fine dell'elaborazione, senza comunicazioni intermedie.
- **Multiprogrammazione**: scheduling dei job basato su un corretto *spooling*. Ogni processo rimane in esecuzione fintanto che non genera un interrupt (eccezione). L'I/O viene attivato contemporaneamente al *context switch*.
- **Time-sharing (multitasking)**: l'alternanza dei processi nella CPU è assegnata a una condivisione equa del tempo di CPU tra i processi. Si usa un **timer**, cioè un registro scalante impostato dal sistema operativo che genera un interrupt.



Condivisione spaziale della RAM

Ogni processo ha accesso solo alla porzione di memoria a lui riservata dal sistema operativo.



Astrazione

Semplificazione dell'uso dell'elaboratore (facilità di programmazione)

- **System call**: esplicita richiesta da parte del programma di accedere a determinati servizi del sistema operativo. E' l'**interfaccia tra processo e sistema operativo**.
- **Trap**: routine eseguite dal sistema operativo a seguito di un **interrupt software**, dovuto a:
 - un errore nella computazione;
 - il tentativo di eseguire un comando privilegiato essendo in modalità utente.

L'obiettivo è alzare il più possibile il **livello di astrazione** dei componenti del sistema di elaborazione.

S.O. : Funzioni

Error detection

Il sistema deve essere in grado di **rilevare errori** nella CPU o nei dispositivi, come:

- la divisione per 0;
- la fine della carta;
- il guasto di una connessione di rete.

Per ogni errore identificato (**interrupt**) è necessario eseguire un'appropriata azione di ricoveri (**trap**).

L'obiettivo è **impedire che il sistema vada in blocco**.

Dual-mode

Una volta avviato il sistema operativo, la CPU può operare in **due modalità**:

1. **User mode**: la CPU sta eseguendo codice di un utente;
2. **Kernel mode** (*supervisor mode, system mode, monitor mode*): la CPU sta eseguendo codice del sistema operativo.

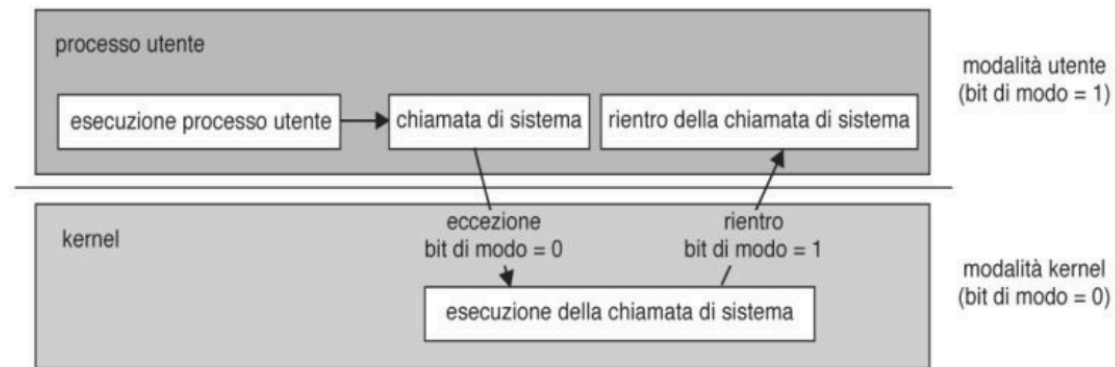
La CPU ha un **mode bit** nel registro di stato (*Process Status Word*, o **PSW**) che indica in quale modo si trova: **kernel (0)** o **user (1)**.

Questa modalità serve a:

- **proteggere i registri** da accessi erronei o internazionali;
- consentire l'**accesso completo all'hardware solo in modo kernel**;
- permettere un **passaggio controllato da modo utente a modo kernel**, tramite:
 - interruzioni;

- chiamate di sistema (*system call*), trap/fault.

L'obiettivo è rendere il sistema quanto più possibile **servo da esecuzioni pericolose**.



Process Management/Memory Management

L'obiettivo è **creare l'ambiente di esecuzione per i programmi**.

Gestione dei processi in relazione alla CPU (esecuzione)

- creazione e terminazione dei processi;
- sospensione e riattivazione dei processi;
- schedulazione dei processi;
- sincronizzazione tra processi;
- gestione di situazioni di stallo (*deadlock*);
- comunicazioni tra processi (memoria condivisa o scambio di messaggi).

Gestione dei processi in relazione alla memoria (allocazione)

- multiprogrammazione;
- allocazione e deallocazione della memoria ai processi;
- caricamento e scaricamento di processi e di loro porzioni in memoria centrale;
- protezione della memoria centrale.

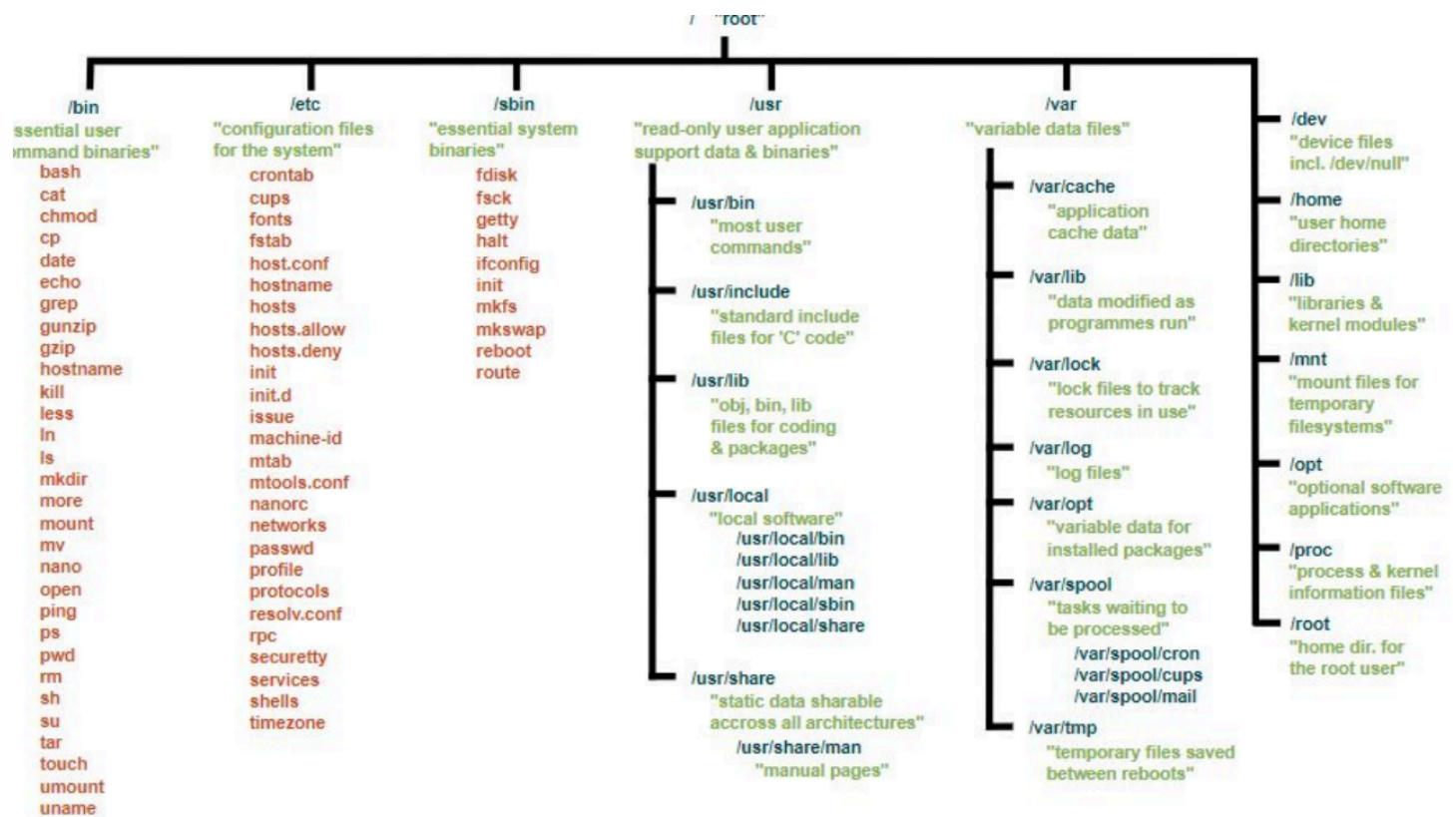
File system

Un **file** è un contenitore di informazioni e dati in formato digitale. Il file system **nasconde all'utente l'organizzazione della memoria** e gestisce:

- file e directory;
- creazione e cancellazione;
- lettura e scrittura;
- copie;
- ricerca;
- salvataggio e ripristino;
- protezione e sicurezza (diritti di accesso).

In Unix **"Everything is a file"**: il file è una pregevole astrazione del sistema operativo.

1. **-** : regular file
2. **d** : directory
3. **c** : character device file
4. **b** : block device file
5. **s** : local socket file
6. **p** : named pipe
7. **l** : symbolic link



Shell/GUI/Transaction

L'obiettivo è rendere il sistema **"user-friendly"** per gli utenti, secondo necessità.

S.O. : Servizi

Boot

L'obiettivo è **permettere al sistema di ripartire in ogni momento.**

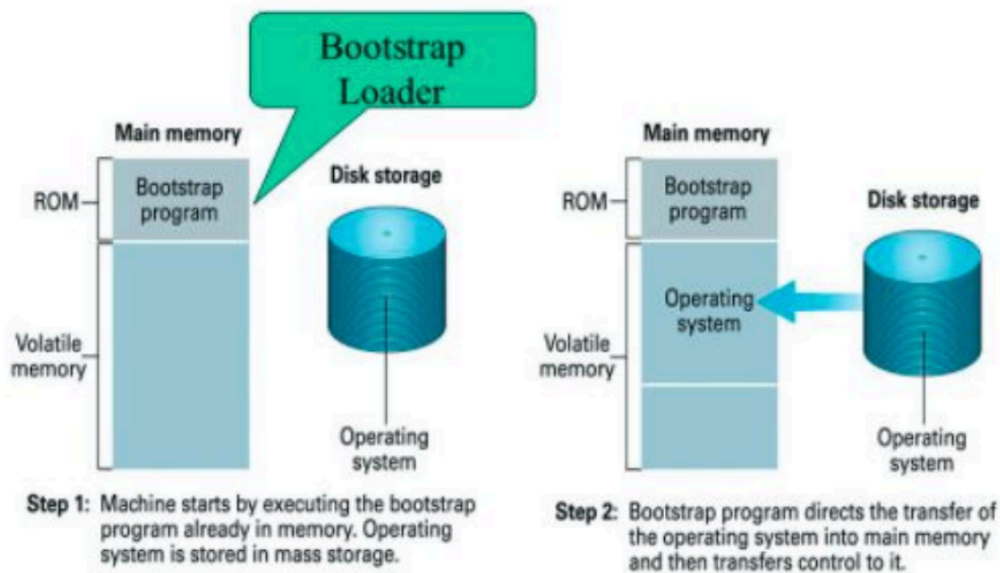
I passi sono:

1. **Accensione.**
2. **Esecuzione del BIOS** (in ROM o EEPROM), *Basic I/O System*. Inizializza tutti i registri della CPU, i controllori delle periferiche e la memoria centrale.
3. **Esecuzione del bootstrap program** (o *bootstrap leader*). Carica il kernel del sistema operativo da una unità di memoria e lo esegue.
4. **Controllo del sistema** da parte del sistema operativo.

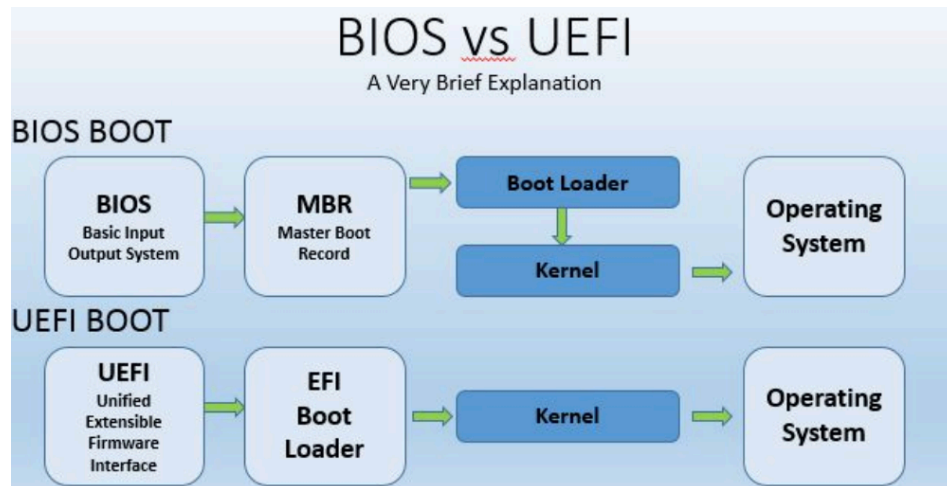
Posizionamento del sistema operativo

In generale, il sistema operativo si trova nel **disco**, in una partizione detta **attiva**, mentre il **bootstrap program** si trova in memoria ROM o EEPROM:

- Il BIOS cerca le periferiche avviabili e dà il controllo al bootstrap program.
- Il bootstrap program si avvale di un frammento di codice, che sa in quale parte del disco si trova il sistema operativo. Questo codice risiede in un blocco del disco a una posizione fissa (tipicamente 0), detto **boot block** o **master boot record (MBR)**.
- La partizione del disco che contiene il boot block è detta **partizione di avvio** (*boot partition*).



- I moderni sistemi **UEFI** hanno una partizione apposita dedicata alle immagini di sistema operativo.



Interrupt handling

Il sistema esegue sequenzialmente le istruzioni. All'arrivo di un interrupt (**I/O guidato da interrupt**), la CPU invoca un “**gestore**” (*routine di servizio*) opportuno attraverso l'interrupt vector:

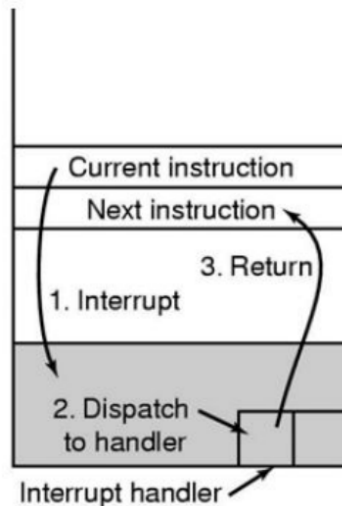
- il **vettore di interrupt** contiene gli indirizzi in memoria di tutte le routine di servizio;
- l'hardware deve **salvare l'indirizzo dell'istruzione interrotta**, in modo tale che, una volta ultimata la gestione dell'interrupt, possa essere ripristinato.

Nota: gli interrupt in arrivo sono **disabilitati mentre un altro interrupt viene gestito**, per evitare che vadano persi.



Passi necessari alla gestione di una interruzione:

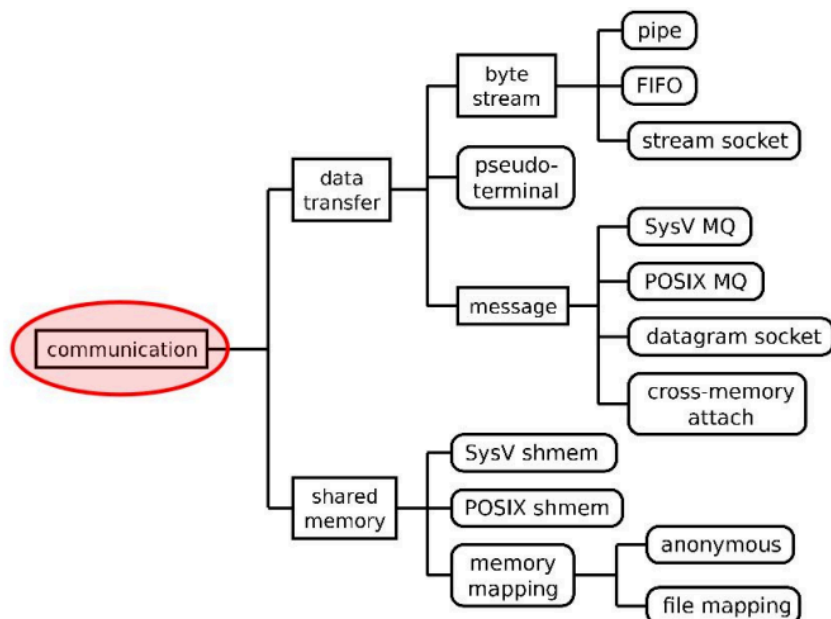
1. La CPU rileva l'interruzione e legge l'indirizzo del dispositivo (device) sul bus:
 - salvataggio dei registro PC e SW (stato della CPU) sullo stack e passaggio in modo kernel.
2. L'indirizzo del dispositivo viene usato come indice nella tabella dei gestori delle interruzioni (interrupt vector);
3. Il gestore selezionato prende il controllo e svolge le operazioni necessarie.
4. Quando il gestore termina:
 - ripristino di PC e PSW, ritorno in modo utente;
 - si esegue l'istruzione successiva a quella interrotta.



IPC (Inter-Process Communication)

L'obiettivo è permettere ai processi di comunicare fra loro:

- **Segnale** (*signals*): per eventi asincroni.
- **Pipe**: per reindirizzare il risultato della elaborazione.
- **Socket**: scambio di datagram.
- **Code di messaggi** (*Message Queue*): per comunicazioni in multicast.
- **Memoria condivisa** (*Shared Memory*): per condividere dati fra processi con trust reciproco.
- **Semaforo** (*Semaphore*): per coordinare processi che lavorano su una stessa risorsa (—> *critical region*).



Signal in Unix

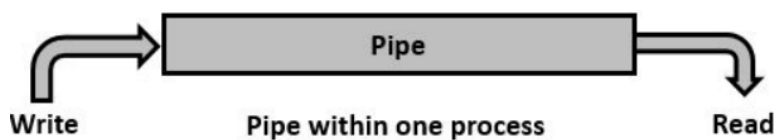
Signal	Value	Action	Comment
SIGHUP	1	Term	Hangup detected on controlling terminal or death of controlling process
SIGINT	2	Term	Interrupt from keyboard
SIGQUIT	3	Core	Quit from keyboard
SIGILL	4	Core	Illegal Instruction
SIGABRT	6	Core	Abort signal from <code>abort(3)</code>
SIGFPE	8	Core	Floating point exception
SIGKILL	9	Term	Kill signal
SIGSEGV	11	Core	Invalid memory reference
SIGPIPE	13	Term	Broken pipe: write to pipe with no readers
SIGALRM	14	Term	Timer signal from <code>alarm(2)</code>
SIGTERM	15	Term	Termination signal
SIGUSR1	30,10,16	Term	User-defined signal 1
SIGUSR2	31,12,17	Term	User-defined signal 2
SIGCHLD	20,17,18	Ign	Child stopped or terminated
SIGCONT	19,18,25	Cont	Continue if stopped
SIGSTOP	17,19,23	Stop	Stop process
SIGTSTP	18,20,24	Stop	Stop typed at terminal
SIGTTIN	21,21,26	Stop	Terminal input for background process
SIGTTOU	22,22,27	Stop	Terminal output for background process

The signals **SIGKILL** and **SIGSTOP** cannot be caught, blocked, or ignored.

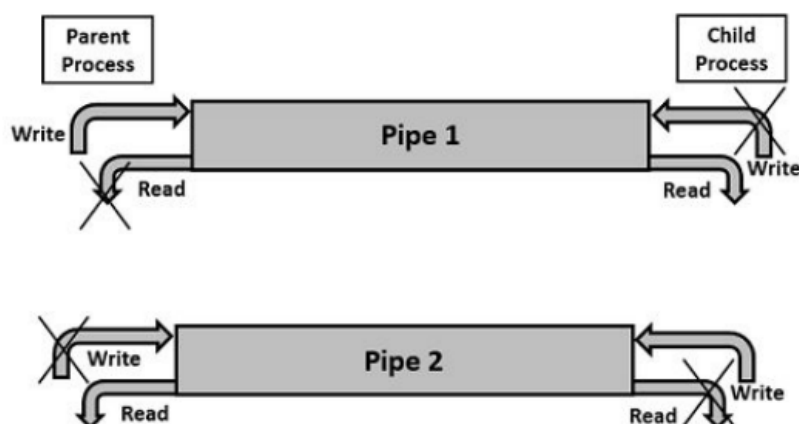
Pipe (datagram FIFO) in Unix

L'obiettivo è **permettere a 2 processi di scambiarsi informazioni fra loro**.

La **pipe** è una comunicazione **one-way**: un flusso (*stream*) **unidirezionale**. Si scrive (write) a un'estremità e si legge (read) dall'altra.



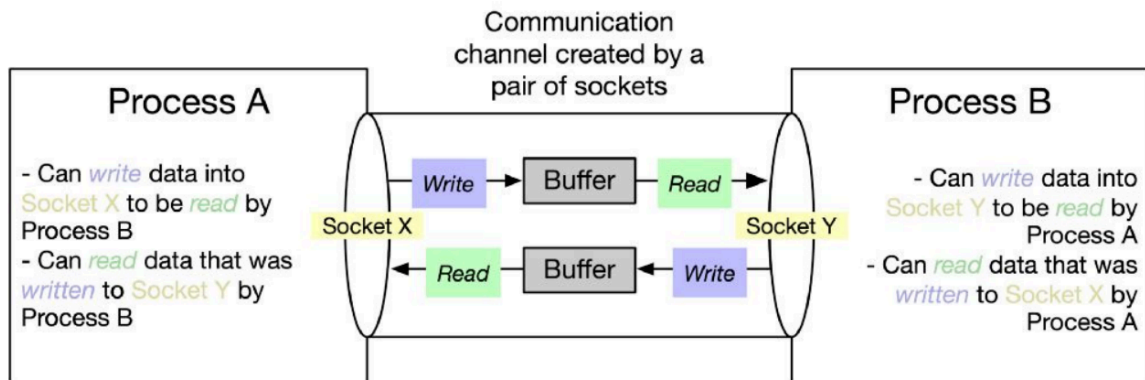
Combinando **2 pipe** si ottiene una comunicazione in entrambe le direzioni tra il processo padre (*parent process*) e il processo figlio (*child process*).



Socket (stream/datagram) in Unix

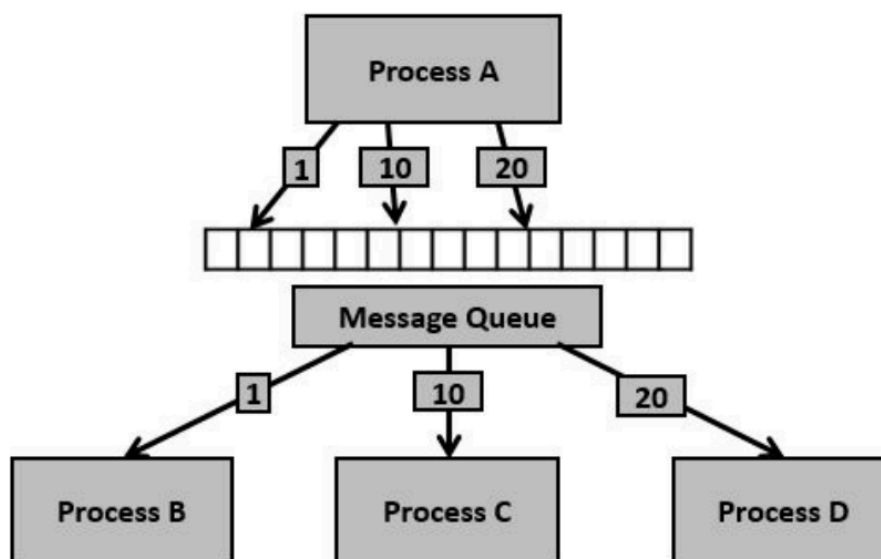
L'obiettivo è **permettere a 2 processi di scambiarsi informazioni fra loro**.

La comunicazione è one-way: un **flusso** (*stream*) o **blocchi di dati** (*datagram*) **unidirezionale**.



Multicast Queue in Unix

L'obiettivo è **permettere a molteplici processi di scambiarsi informazioni non riservate**.

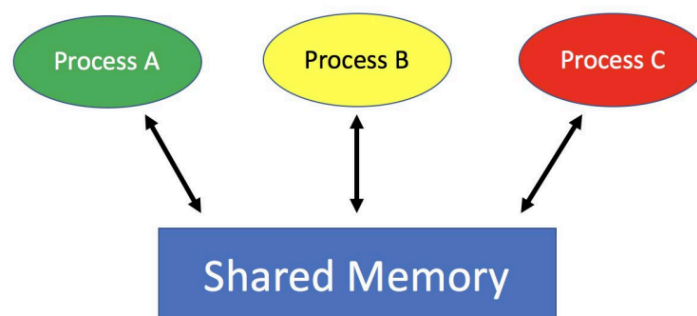


Shared Memory

La **shared memory** è la memoria condivisa fra processi. L'obiettivo è **allocare porzioni di memoria condivisa fra processi**.

I parametri sono:

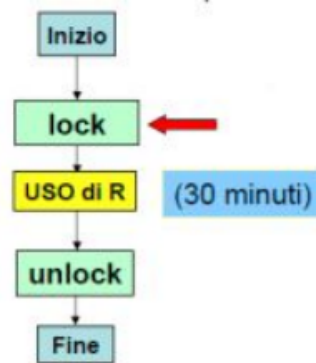
- **SHMMAX**: massima dimensione (in byte) di un singolo segmento di memoria condivisa;
- **SHMMNI**: numero massimo di segmenti di memoria condivisa presenti sul sistema;
- **SHMALL**: numero totale di pagine di memoria condivisa.



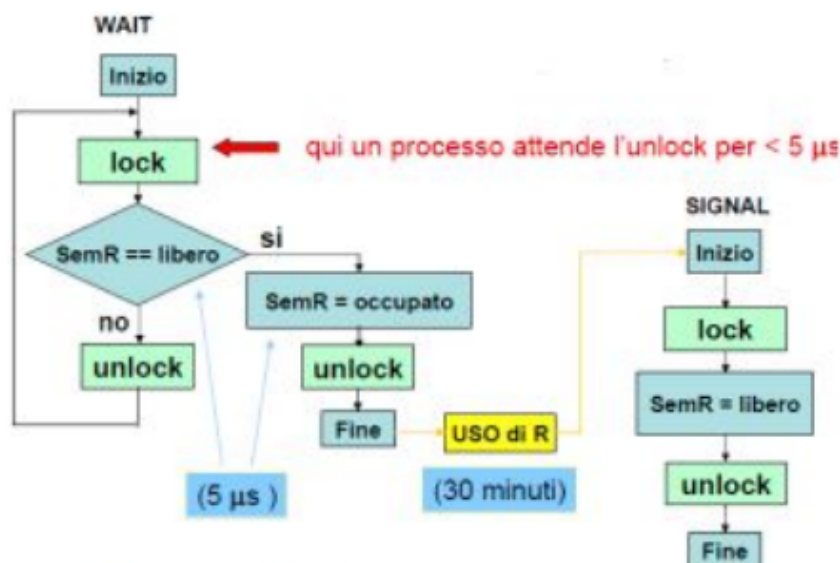
Semaphore in Unix

L'obiettivo è **proteggere una risorsa da accessi concorrenti** e risolvere il problema in modo efficiente in un ambiente multiprogrammato,

- **Sezione critica:** operazioni (istruzioni eseguite da un processo) su variabili di stato di una risorsa comune.
- **Busy waiting:** il processo esegue NOP in attesa che la risorsa bloccata (lock()) venga liberata dal processo che la usa (unlock()).



- **Semaforo:** meccanismo di gestione di una sezione critica, che fa uso di wait() (al posto di lock()) e signal() (al posto di unlock()):
 - la risorsa è guidata da un semaforo che inizialmente vale 1, per indicare lo stato di risorsa libera;
 - il valore del **semaforo binario** è un indicatore che indica se la risorsa è **libera** (1) o **occupata** (2).



System Calls

L'obiettivo è **offrire ai processi le funzionalità offerte dal sistema tramite una interfaccia semplice**.

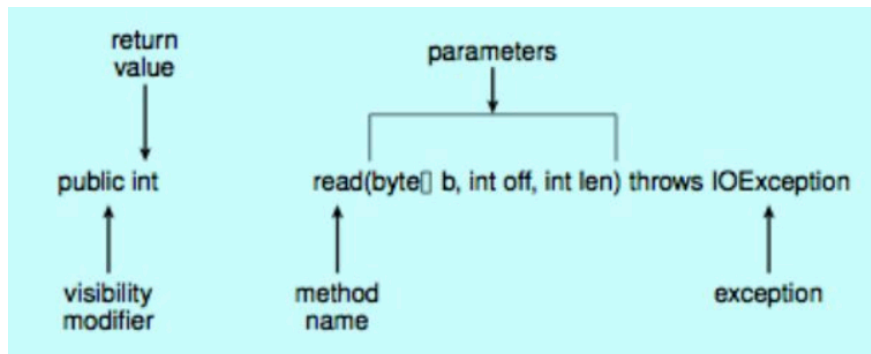
Una chiamata di sistema è l'interazione elementare tra un programma utente e il sistema operativo. Con le chiamate di sistema i programmi utente accedono ai **servizi del sistema operativo**, disponibili come istruzioni in:

- linguaggio assembler;
- linguaggi di programmazione di più alto livello come C e C++, che permettono alla chiamate di sistema di avvenire direttamente.

Java non lo permette:

- poiché una chiamata di sistema è specificata dal sistema operativo e da come risulta un codice specifico della piattaforma;
- tuttavia, se richiesto, è possibile attraverso **metodi nativi**; java può richiamare metodi scritti in un altro linguaggio (C o C++) che eseguono la chiamata.

Il **programmatore** tipicamente **non** invoca le **chiamate di sistema**, ma delle funzioni messe a disposizione da un **API**. Ad esempio, il metodo Java `read()`.

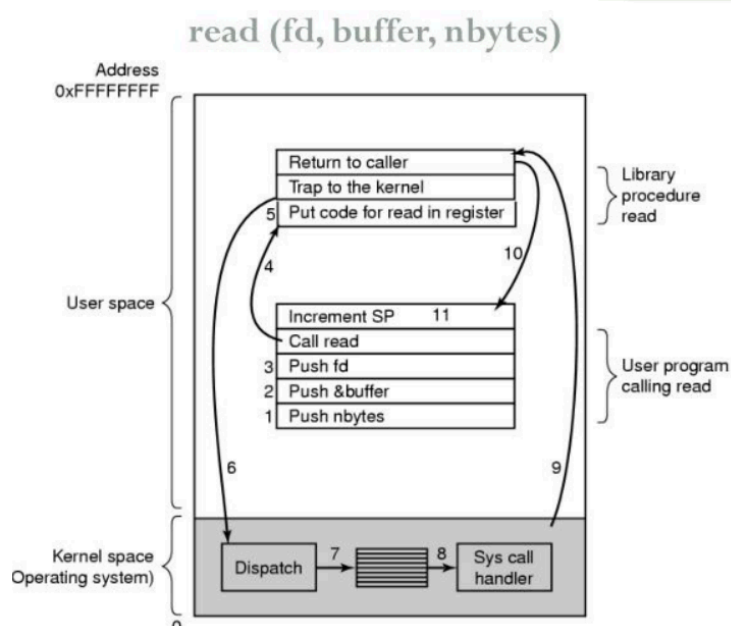


Le chiamate di sistema possono richiedere lo **scambio di informazioni (parametri)** tra un programma e il sistema operativo. Lo scambio dei parametri avviene in generale in **3 modi**:

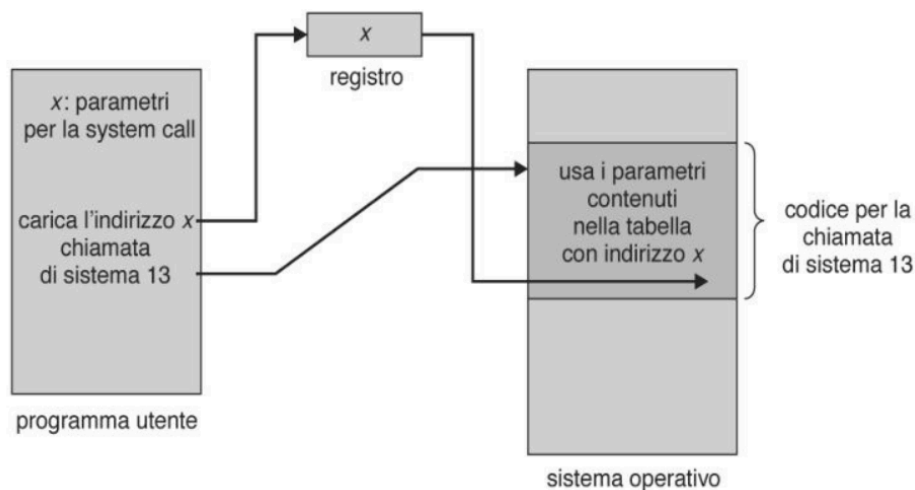
- **Attraverso registri**: attuabile solo se il numero dei parametri non supera quello dei registri.
- **Attraverso una memoria provvisoria detta pila (stack)**: i parametri sono posti dal programma e prelevati dal sistema operativo. Non limita il numero o la lunghezza dei parametri.
- **Attraverso una tabella in memoria**: l'indirizzo X della tabella viene passato come parametro in un registro.

Dettaglio dell'esecuzione di `read (fd, buffer, nbytes)`

- 1-3: salvataggio dei parametri sullo stack.
- 4: chiamata della funzione di libreria `read`.
- 5: caricamento del codice della system call in un registro fissato **Rx**.
- 6: esecuzione della **TRAP**: passaggio in *kernel mode* e salto al codice del *dispatcher*.
- 7-8: selezione della system call secondo il codice in Rx e invocazione del gestore appropriato.
- 9: ritorno alla funzione di libreria: ripristino della *user mode* e caricamento del program counter (**PC**).
- 10-11: ritorno al codice utente (nel modo usuale) e incremento dello stack pointer (**SP**) per rimuovere i parametri della chiamata a `read`.



Attraverso una tabella in memoria, e l'indirizzo x della tabella viene passato come parametro in un registro.



Identification / Authentication / Authorization / Accounting

L'obiettivo è **consentire l'accesso al sistema e alle sue risorse solo agli utenti che ne hanno diritto**.

I passi sono quattro:

1. **Identification.**
2. **Authentication:** si basa su qualcosa che l'utente sa (something you know), possiede (something you own) o è (something you are).
3. **Authorization:** permette l'accesso alla risorsa (resource).
4. **Accountability.**

User Authentication / Authorization

Il file `/etc/passwd` ha tipicamente permessi di file system che gli permettono di essere **leggibile da tutti gli utenti del sistema** (world-readable), anche se può essere modificato solo dal superutente o utilizzando alcuni comandi privilegiati a scopo speciale.

Ogni riga ha 7 campi (esempio: `nish:x:501:501:Nishant Rastogi:/home/nish:/bin/bash`):

1. **Username** (login name);
2. **Password** (la «x» indica che è nel file `/etc/shadow`);
3. **UID** (User Identifier);
4. **GID** (Group Identifier);
5. **General Electric Comprehensive Operating Supervisor** (nome completo, edificio, stanza, telefono d'ufficio, telefono di casa, indirizzo email, ecc.);
6. **Home directory**;
7. **Shell** di avvio a ogni nuova connessione;

L'obiettivo è consentire l'accesso al sistema solo agli utenti che ne hanno diritto.

Accounting

L'obiettivo è **registrare gli accessi al sistema e poterli rendicontare** (se necessario).

- **Windows:** Security Log.
- **Linux:** `/var/log/auth.log`.
- **File di rendicontazione:** `/var/log/utmp`, `btmpt`, `wtmp`.

File Authorization

L'obiettivo è consentire l'accesso al sistema e alle sue risorse solo agli utenti che ne hanno diritto. In Unix i permessi (ad es. `drwxr-xr-x`) si leggono così:

- **tipo di file:**
 - `d`: directory;
 - `-`: regular file;
 - `l`: symbolic link.
- **classi di permessi:** user, group, others.

- **flag di permesso:**

- r: readable;
- w: writable;
- x: executable;
- -: disabled.

Anche in Windows i permessi si impostano per gruppo o utente dalla scheda Security delle proprietà (Allow / Deny).

I Processi

Un **processo** è un programma in esecuzione:

- indipendente dagli altri programmi;
- che non deve interferire con gli altri programmi;
- che trae giovamento dalla gestione ottimizzata delle risorse;
- che deve sottostare all'allocazione e alla condivisione ordinata delle risorse rispetto a spazio e tempo.

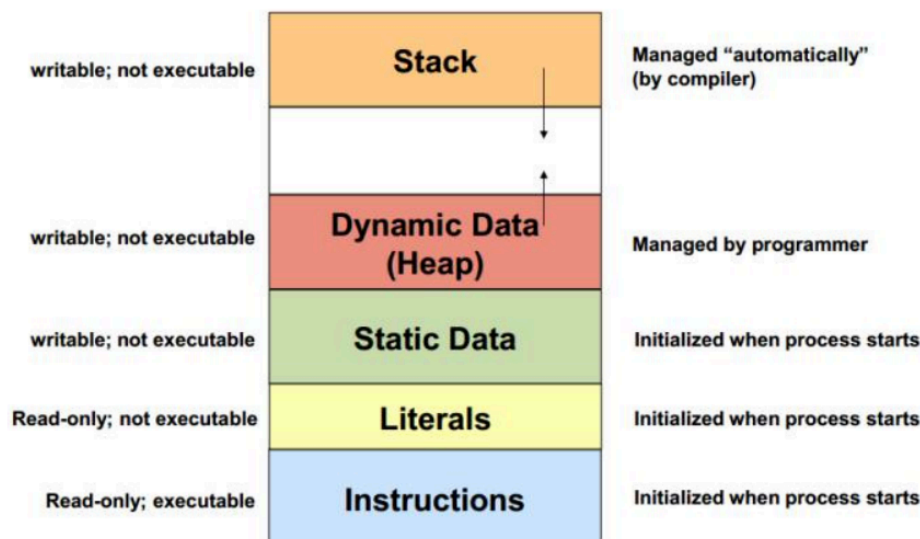
Differenze tra processo e programma:

- **Processo** = esecuzione delle istruzioni → entità attiva. Comprende:
 - contatore di programma;
 - valori delle variabili;
 - risorse in uso.
- **Programma** = lista di istruzioni → entità passiva.

Due processi associati allo stesso programma sono due differenti **istanze** di esecuzione dello stesso codice.

La **memoria occupata** dal processo è composta da:

- **Instructions:** codice del programma (puntato dal *Program Counter*, **PC**);
- **Literals:** parametri del programma;
- **Static data:** variabili globali in memoria centrale;
- **Heap** (mucchio): dati dinamici;
- **Stack** (pila): funzioni (metodi), variabili locali, indirizzi di rientro.



Lo **stack** è una pila di dati che tipicamente cresce verso il basso:

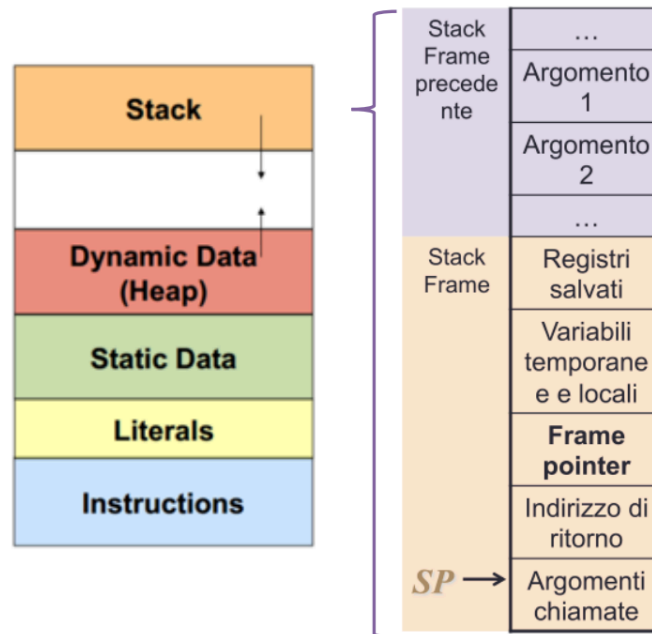
- I record di attivazione delle funzioni (stack frame) vengono inseriti (pusher) e rimossi (popped) dallo stack.
- Lo task frame contiene dati relativi ad una funzione:
 - parametri;

- variabili locali;
- dati necessari per ripristinare il frame precedente.
- Viene puntato dal **Frame Pointer (FP)**.

Lo **heap** è una pila di dati che tipicamente cresce verso l'alto:

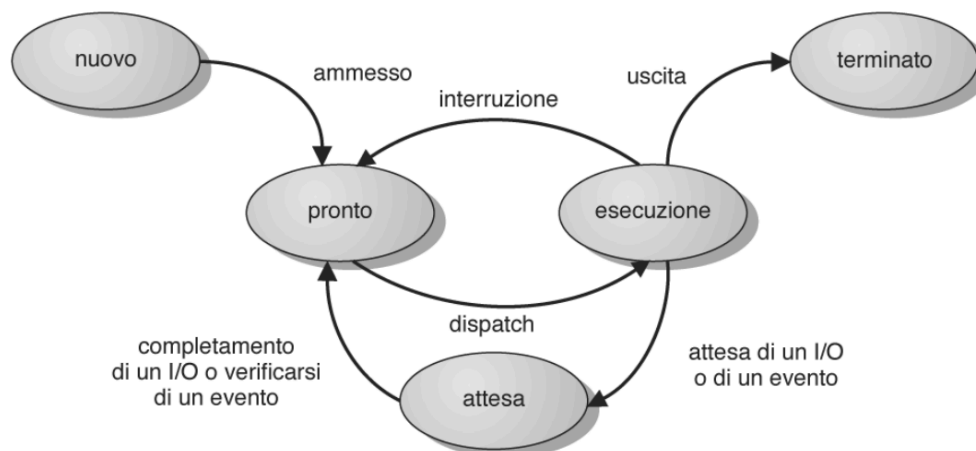
- il programmatore si occupa di come utilizzare l'heap (con il SO).
- Si deve occupare di allocare (malloc C) e deallocare la memoria.

Se stack e heap **collidono** si verificano **errori**: esistono chiamate apposta per ridimensionarli.



I possibili **stati di uso del processore** da parte del processo sono:

- **Nuovo** (*new*): il processo è stato creato;
- **In esecuzione** (*running*): le istruzioni vengono eseguite;
- **In attesa** (*waiting*): il processo sta aspettando il verificarsi di qualche evento;
- **Pronto all'esecuzione** (*ready*): il processo è in attesa di essere assegnato a un processore;
- **Terminato** (*terminated*): il processo ha terminato l'esecuzione.



PCB

Process Control Block (PCB): Struttura dati del **kernel** che mantiene le informazioni sul processo

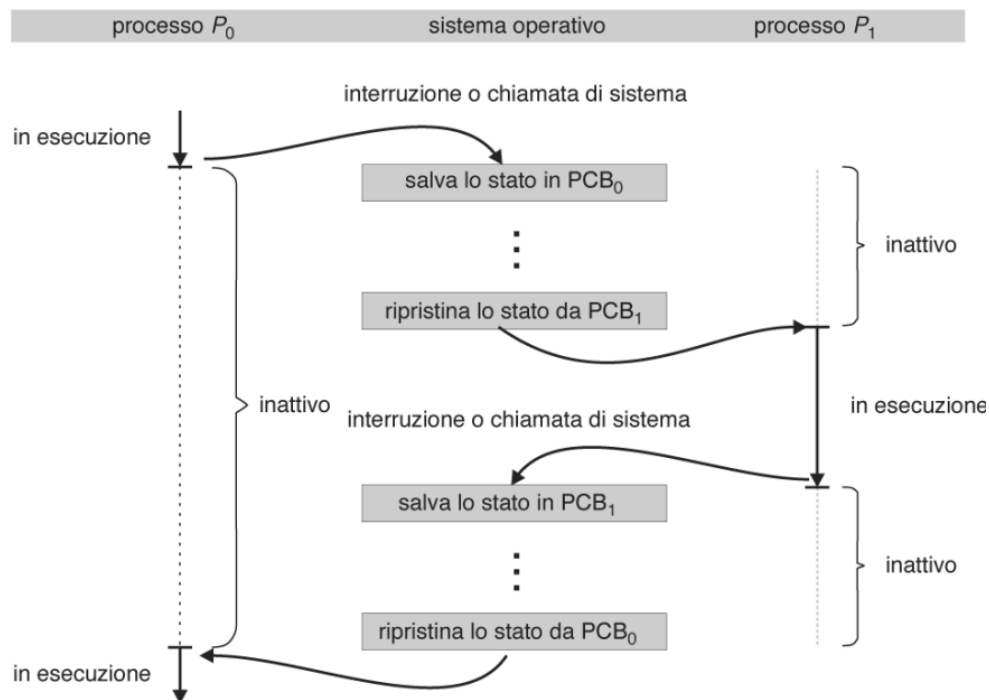
Context Switch

Il **context switch** è il passaggio della CPU da un processo a un altro. Consiste in due operazioni:

- la **sospensione** del processo in esecuzione;
- il **caricamento** del nuovo processo.

Inoltre:

- il tempo, per il cambio di contesto è puro **tempo di gestione del sistema**: non vengono compiute operazioni utili per la computazione di processi;
- i tempi (10ms) per i cambi di contesto dipendono dal **supporto hardware** (as esempio registri disponibili e/o registri dedicati).



Unix SysCall: fork(), wait(), exit()

Un **processo in esecuzione** può creare numerosi sottoprocessi usando un'apposita chiamata di sistema (create_process):

- **Processo generante** = **processo padre**;
- **Processo generato** = **processo figlio**.

Ogni processo ha un identificatore univoco **PID** (intero).

In unix si usa la chiamata **fork()**, che crea un nuovo processo:

- si forma un albero di processi;
- inizialmente il figlio è la copia del padre;
- la **fork()** ritorna un valore:
 - 0: figlio;
 - $\neq 0$: padre.
- **exec()** sovrascrive lo spazio degli indirizzi del chiamante caricando un nuovo programma.

Dopo la creazione del figlio, l'esecuzione del processo padre:

- continua in modo concorrente, oppure;
- attende la terminazione del figlio $\rightarrow$ **wait()**;
- la **chiamata fork()** crea un nuovo processo. Il valore di ritorno vale:
 - 0 per il figlio;
 - diverso da 0 per il padre.

La **fork** crea una copia dello spazio degli indirizzi del genitore. Entrambi, padre e figlio, continuano l'esecuzione della chiamata successiva alla fork.

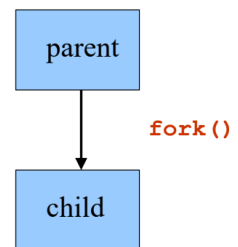
La chiamata **wait(NULL)** blocca il chiamante fino alla terminazione di un figlio (il primo che termina).

waitpid(..., int pid, ...) permette di attendere la terminazione di uno specifico figlio.

Creazione di nuovi processi

Il meccanismo alla base della creazione di nuovi processi è il seguente:

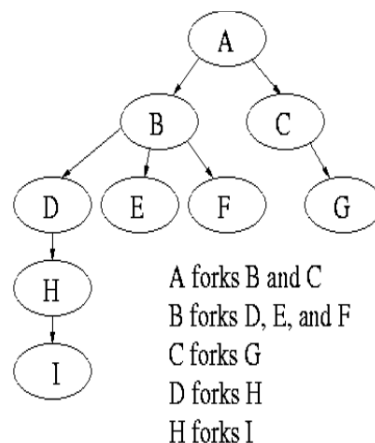
- un processo esegue **fork** per creare un processo figlio;
- padre e figli vengono eseguiti in modo concorrente;
- il processo figlio è un **duplicato** del processo padre.



Creazione di processi

Dopo una fork, sia il padre sia il figlio continuano a essere seguiti, e ciascuno può a sua volta creare altri processi con fork:

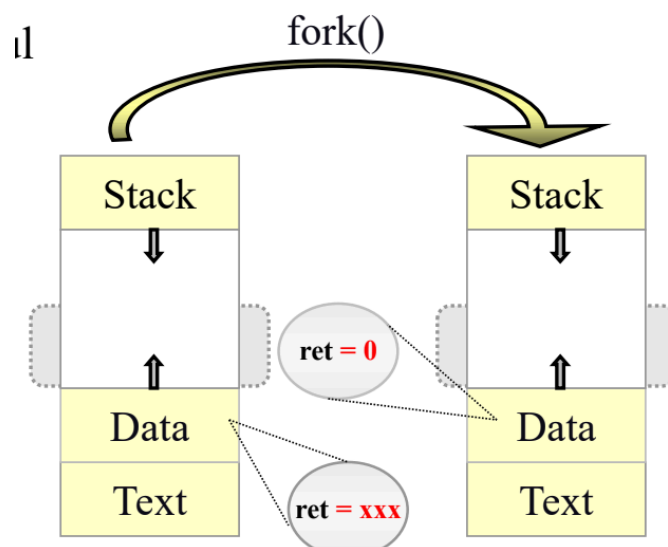
- Ne risulta un **albero di processi**: la radice dell'albero è un processo speciale creato dal sistema operativo durante l'avvio.
- Un processo può scegliere di attendere la terminazione dei figli. Ad esempio, se C esegue la chiamata di sistema wait(), resta bloccato finché G non termina.



Fork()

Il processo corrente si divide in 2 processi: **padre** e **figlio**. La fork():

- ritorna -1 se non ha successo;
- ritorna 0 nel figlio;
- ritorna l'identificatore del figlio nel padre.



Il processo figlio eredita dal padre:

- una copia identica della memoria;
- i registri della CPU;
- tutti i file che sono stati aperti dal padre.

L'esecuzione prosegue in modo **concorrente** con l'istruzione successiva alla chiamata di sistema fork.

Il **contesto di esecuzione (PCB)** del processo figlio è una **copia del contesto del padre** al momento della chiamata.

Terminazione del processo

La **terminazione del processo** si divide in:

- normale;
- anomala.

La **terminazione normale** avviene dopo l'ultima istruzione, tramite la chiamata **exit**:

- il figlio può restituire un valore di stato (di solito un intero) al padre, ad esempio tramite la chiamata **wait()**;
- le risorse del processo sono deallocate dal sistema operativo:
 - file aperti;
 - memoria fisica e virtuale;
 - aree di memoria per I/O.

Nella **terminazione in caso di anomalia**, il padre può terminare l'esecuzione di uno dei suoi figli per varie ragioni:

- eccessivo uso di una risorsa;
- compito non più necessario.

Terminazione a cascata:

- Se il padre sta terminando, alcuni sistemi operativi non permettono a un processo figlio di proseguire.
- In unix la terminazione a cascata non esiste: i processi figli vengono adottati dal processo **init**.

La mancata terminazione tramite la chiamata **wait()** porta a un processo "**zombie**": un processo la cui elaborazione è terminata, ma a cui il padre non ha dato il consenso per la terminazione.

System calls: exit(), terminazione ordinata

Per terminare l'esecuzione, un figlio può chiamare **exit(status)**. Questa chiamata di sistema:

- salva il risultato, cioè l'argomento di **exit**;
- esegue tutte le funzioni specificate con **atexit(fun)** e **on_exit(fun)**;
- scarica gli stream con **fflush()**;
- chiude tutti i file e le connessioni aperte (non quelli condivisi con altri processi);
- chiama **_exit(status)**.

System calls: _exit(), fine dell'esecuzione

Per finire l'esecuzione, un figlio può chiamare **_exit(status)**. Questa chiamata di sistema:

- salva il risultato, cioè l'argomento di **exit**;
- dealloca la memoria;
- se il processo ha figli in esecuzione, li assegna a **init**;
- controlla se il padre è vivo:
 - se il padre è vivo, conserva il valore del risultato finché il padre non lo richiede (con **wait**). In questo caso il processo figlio non muore davvero, ma entra in uno stato **zombie/defunct**;
 - se il padre non è vivo, il figlio termina (muore).

System calls: wait(), attesa della fine di un figlio

Il padre può voler attendere la fine dei figli (ad esempio una shell che aspetta il completamento delle operazioni):

- **Attesa di un figlio qualsiasi: wait()**
 - si blocca finché un figlio termina;
 - ritorna il processo ID del processo figlio;
 - oppure ritorna -1 se non esistono figli (cioè sono già terminati).
- **Attesa di un figlio specifico: waitpid()**
 - si blocca finché termina il figlio con un particolare process ID.

```
#include <sys/types.h>
#include <sys/wait.h>

pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);
```